

File permissions in Linux

Project description

As a security professional, I was tasked with ensuring the users from the research team of my organization are authorized with appropriate file permissions on the organization's computer system. I had to examine the existing permissions within the `/home/researcher2/projects` directory for the user `researcher2` and update any misconfigured permissions to help keep the system secure.

Checking file and directory details

I began by navigating to the `projects` directory, where I used the `ls -la` command to get a detailed list of all the files in that directory along with their permissions:

```
researcher2@07e916aaf63f:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Aug 28 13:29 .
drwxr-xr-x 3 researcher2 research_team 4096 Aug 28 14:18 ..
-rw--w---- 1 researcher2 research_team  46 Aug 28 13:29 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Aug 28 13:29 drafts
-rw-rw-rw- 1 researcher2 research_team  46 Aug 28 13:29 project_k.txt
-rw-r----- 1 researcher2 research_team  46 Aug 28 13:29 project_m.txt
-rw-rw-r--  1 researcher2 research_team  46 Aug 28 13:29 project_r.txt
-rw-rw-r--  1 researcher2 research_team  46 Aug 28 13:29 project_t.txt
```

The `ls` command in Linux is used to display the files inside a directory. The `-la` perimeter which follows it is a combination of `-a`, which ensures that hidden files are displayed as well; and `-l`, which is used to display detailed information including the permissions string.

The permissions string

The “permissions string” consists of the first ten characters of each file's details after running the detailed listing command. The first character of this string identifies the type of file (`d` = directory, `-` = regular file). The remaining nine characters are divided into groups of three, and each group identifies the permissions for each of the three owner types in a Linux system, “user”, “group”, and “other”, respectively. The three characters of each group are the types of permission granted to each owner: the first character is for “read” permission, second one for “write”, and third character is for the “execute” permission. The presence of a character means that the related permission is granted to that group, and a hyphen means that permission is not granted.

For example, the permission string for the file `project_k.txt` is `-rw-rw-rw-`. Here, the first hyphen means that this is a regular file and not a directory. After this, the three groups each are `rw-`, which means that all three owner types can read and write to the file but cannot execute it (`r` = read, `w` = write, `-` = cannot execute.)

Changing file permissions

The organization does not allow others to have write access to any files, but we can see from the detailed listing that `project_k.txt` currently allows others to write to the file. Furthermore, the file `project_m.txt` is supposed to be a restricted file, which means that only the user should have permission to read and write to it, but as of now the group can read it too. I changed the permissions on these files by running these two commands:

```
chmod o-w project_k.txt
```

```
chmod g-r project_m.txt
```

```
researcher2@07e916aaf63f:~/projects$ chmod o-w project_k.txt
researcher2@07e916aaf63f:~/projects$ chmod g-r project_m.txt
researcher2@07e916aaf63f:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Aug 28 13:29 .
drwxr-xr-x 3 researcher2 research_team 4096 Aug 28 14:18 ..
-rw--w---- 1 researcher2 research_team  46 Aug 28 13:29 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Aug 28 13:29 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Aug 28 13:29 project_k.txt
-rw----- 1 researcher2 research_team  46 Aug 28 13:29 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 28 13:29 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 28 13:29 project_t.txt
```

`chmod` is the Linux command to change permissions on a file. For the first command, the perimeters `o-w` mean that we want to remove write permission from others (`-` to remove a permission, `+` to add a permission). For second command, the perimeters `g-r` are used to remove read permission from the group file belongs to. The last perimeter in both commands specifies the file on which these changes are needed. I used `ls -la` again at the end to verify if the permissions were changed correctly.

Changing file permissions on a hidden file

In Linux, a hidden file will always have a name beginning with a period. In the `projects` directory, `.project_x.txt` is a hidden file that has been archived and should not be written to by anyone, but the user and the group should still have permission to read this file. As of now, both the user and the group have write access to file, and the group is missing the read access. I corrected this by using `chmod` in a different way this time:

```
chmod 440 .project_x.txt
```

```
researcher2@07e916aaf63f:~/projects$ chmod 440 .project_x.txt
researcher2@07e916aaf63f:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Aug 28 13:29 .
drwxr-xr-x 3 researcher2 research_team 4096 Aug 28 14:18 ..
-r--r----- 1 researcher2 research_team  46 Aug 28 13:29 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Aug 28 13:29 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Aug 28 13:29 project_k.txt
-rw----- 1 researcher2 research_team  46 Aug 28 13:29 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 28 13:29 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 28 13:29 project_t.txt
```

This format for `chmod` is called the “numeric mode,” and it allows us to set permissions for each of the three owner types simultaneously using a three-digit numeric code (each digit representing each of the three owner types). This mode gives a numeric value to each permission; read is given 4, write is given 2, and execute is given 1. To set multiple permissions, we just need to add their values. For example, if we want to give all permissions to user, only read and write to the group, and only read permission to others, our numeric code will be 764:

- 7: For users, read = 4, write = 2, execute = 1. $4 + 2 + 1 = 7$,
- 6: For group, read = 4, write = 2. $4 + 2 = 6$,
- 4: For others, read = 4. Since read is the only permission, we just put 4.

In case an owner type needs no permissions, we use a 0 in its place. In our case, we need to grant read permissions to user and group, and no permissions to others, therefore we used `440` as the code. After running the command, I once again ran `ls -la` to verify the changes.

Changing directory permissions

As stated earlier, directories are identified in a Linux system if the first character of their permission string is a ‘d.’ The `projects` folder has the directory `drafts`, which has all permissions for user and only execute permission for group. Enabling execute permission for a particular owner on directories means giving access to the contents of that directory to that owner. The organization’s policy states that the directory `drafts` and its contents should only be accessed by the user `researcher2`. Right now, the group `research_team` can also access the directory. I fixed this by running `chmod` in the following way: `chmod g=- drafts`.

```
researcher2@07e916aaf63f:~/projects$ chmod g=- drafts
researcher2@07e916aaf63f:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Aug 28 13:29 .
drwxr-xr-x 3 researcher2 research_team 4096 Aug 28 14:18 ..
-r--r----- 1 researcher2 research_team  46 Aug 28 13:29 .project_x.txt
drwx----- 2 researcher2 research_team 4096 Aug 28 13:29 drafts
-rw-rw-r-- 1 researcher2 research_team  46 Aug 28 13:29 project_k.txt
-rw----- 1 researcher2 research_team  46 Aug 28 13:29 project_m.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 28 13:29 project_r.txt
-rw-rw-r-- 1 researcher2 research_team  46 Aug 28 13:29 project_t.txt
```

This `chmod` format can be used to set permissions for a particular owner at once. Here, the character before `=` identifies the owner (u for user, g for group, o for others). The character(s) following `=` are the permissions for that owner (r would mean read, wx would mean write and execute, and a hyphen (-) would mean no permissions). Since removing the execute permission would leave group with no permissions to drafts, I used `g=-` as the parameter. Using `ls -la` afterwards confirmed the change.

Summary

I was able to use multiple Linux commands to examine the files inside projects directory and their permissions, look for any misconfigurations, and update the permissions to match the level of authorization my organization wants in order to keep the system used by members of research team secure.